BİL377

Ödev 1 — Dinamik Programlama

Partition Equal Subset Sum
(Eşit Toplamlı Alt Kümelere Bölme)

Konu No: 6

Ad Soyad
Nuraziz Taymusaev

1. Problemin Tanımı

Pozitif tam sayılardan oluşan bir **nums** dizisi veriliyor. Dizinin elemanlarını, her eleman tam olarak bir kümede yer alacak şekilde, **toplamları eşit olan iki alt kümeye** bölmenin mümkün olup olmadığı soruluyor.

Girdi: n uzunluğunda pozitif tam sayı dizisi, örn. `nums = [1, 5, 11, 5]`.

Çıktı: `True` (bölünebilir) veya `False`. Örnekte cevap True'dur: `[11]` ve `[1, 5, 5]`, her ikisinin toplamı 11.

Temel gözlem: Toplam S ise, iki eşit küme varsa her birinin toplamı S/2 olmalıdır. Dolayısıyla S tek ise cevap doğrudan False'tur. S çift ise problem, **toplamı tam olarak S/2 olan bir alt küme var mı?** sorusuna (Subset Sum) indirgenir: böyle bir küme seçilirse geri kalan elemanların toplamı da otomatik olarak S/2 olur.

Kaba kuvvet çözümü 2^n alt kümeyi dener. Dinamik programlama ile, aynı alt problemler tekrar tekrar çözülmediği için süre `O(n · S/2)`'ye iner.

2. DP Mantığı

2.1 Durumun anlamı

`dp[i][j]` = ilk **i** eleman (a_1, …, a_i) arasından bazıları seçilerek toplamı tam olarak **j** olan bir alt küme oluşturulabiliyorsa **True**, aksi hâlde **False**. Burada `0 ≤ i ≤ n`, `0 ≤ j ≤ hedef = S/2`.

2.2 Başlangıç koşulları

- `dp[i][0] = True` — boş küme seçilerek toplam 0 her zaman elde edilir.
- `dp[0][j] = False (j > 0)` — hiç eleman yokken pozitif bir toplam elde edilemez.

2.3 Geçiş bağıntısı

i. eleman a_i için iki seçenek vardır: onu **almamak** ya da ($j \geq a_i$ ise) **almak**.

```
dp[i][j] = dp[i-1][j] OR ( j ≥ a_i AND dp[i-1][j - a_i] )
```

Almazsak j toplamını ilk i−1 eleman ile yapmamız gerekir; alırsak geriye kalan $j - a_i$ toplamını ilk i−1 eleman ile yapmamız gerekir. Bir eleman yalnızca bir kez kullanılabildiği için her iki durumda da bir önceki satıra (i−1) bakılır.

2.4 Cevap

S tek ise `False`; aksi hâlde `dp[n][S/2]`.

2.5 Hesaplama sırası

Tablo satır satır (i = 1, 2, …, n), her satırda soldan sağa (j = 1, …, hedef) doldurulur. Her hücre yalnızca bir üst satırdaki iki hücreye bağlı olduğundan, bu sırada ihtiyaç duyulan değerler her zaman önceden hesaplanmış olur.

3. Örnek Üzerinde DP Tablosu

`nums = [1, 5, 11, 5]`, S = 22 (çift), hedef = 11. Satırlar: kullanılan eleman sayısı i; sütunlar: hedef toplam j. Yeşil = True, kırmızı = False.

i \ j	0	1	2	3	4	5	6	7	8	9	10	11
0: {}	T	F	F	F	F	F	F	F	F	F	F	F
1: a=1	T	T	F	F	F	F	F	F	F	F	F	F
2: a=5	T	T	F	F	F	T	T	F	F	F	F	F
3: a=11	T	T	F	F	F	T	T	F	F	F	F	T
4: a=5	T	T	F	F	F	T	T	F	F	F	T	T

Satırların açıklaması (hesaplanma sırasıyla):

i = 0: yalnızca dp[0][0] = T (taban durum).

i = 1 (a=1): dp[1][1] = dp[0][1] OR dp[0][0] = F OR T = **T**. Diğer j'ler için 1 alınsa bile dp[0][j−1] = F olduğundan F. Erişilebilen toplamlar: {0, 1}.

i = 2 (a=5): j = 5: dp[1][5] OR dp[1][0] = F OR T = **T**; j = 6: dp[1][6] OR dp[1][1] = F OR T = **T**. Örneğin j = 8: dp[1][8] OR dp[1][3] = F OR F = F. Erişilebilen toplamlar: {0, 1, 5, 6}.

i = 3 (a=11): j < 11 için eleman alınamaz, satır kopyalanır. j = 11: dp[2][11] OR dp[2][0] = F OR T = **T**. Erişilebilen: {0, 1, 5, 6, 11}.

i = 4 (a=5): j = 10: dp[3][10] OR dp[3][5] = F OR T = **T**; j = 11: dp[3][11] = T zaten. Erişilebilen: {0, 1, 5, 6, 10, 11}.

Cevap: dp[4][11] = **True** (turuncu çerçeveli hücre).

Geri izleme (alt kümenin bulunması)

dp[n][hedef]'ten başlayıp yukarı çıkılır: dp[i−1][j] = F ise a_i mutlaka seçilmiştir ve j ← j − a_i yapılır; aksi hâlde a_i atlanır. dp[4][11]: dp[3][11] = T → a_4 atla. dp[3][11]: dp[2][11] = F → a_3 = 11 seçildi, j = 0. Sonuç: `[11]` ve `[1, 5, 5]`.

Aynı örnek GIF animasyonunda adım adım gösterilmiştir (aşağıda animasyonun son karesi). GIF, tablo ve programın çıktısı aynı değerleri verir.

Partition Equal Subset Sum — DP

nums = **1** **5** **11** **5** toplam = 22, hedef = 11

dp[i][j] = ilk i eleman ile toplam j yapılabilir mi?
dp[i][j] = dp[i-1][j] OR dp[i-1][j - a_i] (j ≥ a_i ise)



Sonuç: [11] (toplam 11) | [1, 5, 5] (toplam 11)

Çıktı: True

hesaplanan dp[i-1][j] (alma) dp[i-1][j-a] (al) True False geri izleme

4. Python Kodu

Dosya: `partition_equal_subset_sum.py`. 2B tablolu sürüm (tabloyu göstermek ve alt kümeyi geri izlemek için) ve bellek açısından optimize 1B sürüm birlikte verilmiştir; program iki sürümün aynı sonucu verdiğini `assert` ile kontrol eder.

```
"""
Partition Equal Subset Sum (Eşit Toplamlı Alt Küme Bölmesi) — Dinamik Programlama

Problem: Pozitif tam sayılardan oluşan bir dizi verildiğinde, dizinin
toplamları eşit olan iki alt kümeye bölünüp bölünemeyeceğini belirle.

DP durumu:
    dp[i][j] = True  <=>  ilk i elemandan bazıları seçilerek toplam j elde edilebilir
Başlangıç:
    dp[i][0] = True (boş küme), dp[0][j>0] = False
Geçiş:
    dp[i][j] = dp[i-1][j]                          (i. elemanı alma)
            OR (j >= a_i AND dp[i-1][j - a_i])     (i. elemanı al)
Cevap:
    toplam tek ise False, değilse dp[n][toplam/2]

Çalıştırma:
    python3 partition_equal_subset_sum.py              # örnekleri çalıştırır
    python3 partition_equal_subset_sum.py 1 5 11 5     # kendi girdiniz
"""
import sys


def can_partition_table(nums):
    """2B DP tablosunu kurar. (sonuç, tablo, hedef) döndürür."""
    total = sum(nums)
    if total % 2 == 1:
        return False, None, None
    target = total // 2
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = True
    for i in range(1, n + 1):
        a = nums[i - 1]
        for j in range(1, target + 1):
            dp[i][j] = dp[i - 1][j]
            if j >= a and dp[i - 1][j - a]:
                dp[i][j] = True
    return dp[n][target], dp, target


def reconstruct(nums, dp, target):
    """Tablodan geriye giderek toplamı target olan alt kümeyi bulur."""
    subset, j = [], target
    for i in range(len(nums), 0, -1):
        if not dp[i - 1][j]:          # i. eleman olmadan j yapılamıyorsa, o eleman seçilmiştir
            subset.append(nums[i - 1])
            j -= nums[i - 1]
    return subset[::-1]


def can_partition_1d(nums):
    """Bellek optimizasyonlu sürüm: O(hedef) bellek."""
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    dp = [True] + [False] * target
    for a in nums:
        for j in range(target, a - 1, -1):  # sağdan sola: her eleman en fazla bir kez
            dp[j] = dp[j] or dp[j - a]
```

```
    return dp[target]


def print_table(nums, dp, target):
    header = "        j: " + " ".join(f"{j:>2}" for j in range(target + 1))
    print(header)
    for i in range(len(dp)):
        label = "{}" if i == 0 else f"a{i}={nums[i-1]}"
        row = " ".join(" T" if v else " F" for v in dp[i])
        print(f"{label:>10} {row}")


def solve(nums):
    print(f"Girdi: {nums}   toplam = {sum(nums)}")
    ok, dp, target = can_partition_table(nums)
    assert ok == can_partition_1d(nums)
    if dp is None:
        print("Toplam tek sayı -> eşit bölme imkânsız.\nÇıktı: False\n")
        return ok
    print(f"Hedef = toplam / 2 = {target}")
    print_table(nums, dp, target)
    if ok:
        s1 = reconstruct(nums, dp, target)
        s2 = list(nums)
        for x in s1:
            s2.remove(x)
        print(f"Bölme: {s1} (toplam {sum(s1)})  |  {s2} (toplam {sum(s2)})")
    print(f"Çıktı: {ok}\n")
    return ok


if __name__ == "__main__":
    if len(sys.argv) > 1:
        solve([int(x) for x in sys.argv[1:]])
    else:
        for example in ([1, 5, 11, 5], [2, 2, 3, 5], [1, 2, 3, 5]):
            solve(example)
```

5. Çalıştırma Adımları

1. Python 3.8+ kurulu olmalıdır (ek kütüphane gerekmez).
2. Örnekleri çalıştırmak için: `python3 partition_equal_subset_sum.py`
3. Kendi girdiniz için sayıları argüman olarak verin: `python3 partition_equal_subset_sum.py 1 5 11 5`
4. (İsteğe bağlı) GIF'i yeniden üretmek için: `pip install pillow`, sonra `python3 make_gif.py`

6. Örnek Girdiler ve Çıktılar

Üç örnek: [1, 5, 11, 5] → True; [2, 2, 3, 5] → toplam çift (12) olmasına rağmen toplamı 6 olan alt küme olmadığından False; [1, 2, 3, 5] → toplam tek (11) olduğundan doğrudan False. Programın gerçek çıktısı:

```
Girdi: [1, 5, 11, 5]   toplam = 22
Hedef = toplam / 2 = 11
        j:  0  1  2  3  4  5  6  7  8  9 10 11
        {}  T  F  F  F  F  F  F  F  F  F  F  F
      a1=1  T  T  F  F  F  F  F  F  F  F  F  F
      a2=5  T  T  F  F  F  T  T  F  F  F  F  F
     a3=11  T  T  F  F  F  T  T  F  F  F  F  T
      a4=5  T  T  F  F  F  T  T  F  F  F  T  T
Bölme: [11] (toplam 11)  |  [1, 5, 5] (toplam 11)
Çıktı: True
```

```
Girdi: [2, 2, 3, 5]   toplam = 12
Hedef = toplam / 2 = 6
        j:  0  1  2  3  4  5  6
        {}  T  F  F  F  F  F  F
      a1=2  T  F  T  F  F  F  F
      a2=2  T  F  T  F  T  F  F
      a3=3  T  F  T  T  T  T  F
      a4=5  T  F  T  T  T  T  F
Çıktı: False

Girdi: [1, 2, 3, 5]   toplam = 11
Toplam tek sayı -> eşit bölme imkânsız.
Çıktı: False
```

7. Zaman ve Bellek Karmaşıklığı

Değişkenler: **n** = dizideki eleman sayısı, **S** = elemanların toplamı, **T = S/2** = hedef toplam.

Zaman: Tabloda (n+1)·(T+1) hücre vardır ve her hücre O(1) işlemle (iki hücreye bakarak) hesaplanır. Toplamın hesaplanması O(n). Toplam: **O(n · T) = O(n · S)**. Geri izleme O(n).

Bellek: 2B tablolu sürüm **O(n · T)**. Her satır yalnızca bir önceki satıra bağlı olduğundan tek bir dizi yeterlidir: 1B sürüm **O(T)** bellek kullanır. 1B sürümde j **sağdan sola** (T'den a'ya) dolaşılır; böylece dp[j − a] henüz bu elemanla güncellenmemiş (önceki satırın) değeri gösterir ve her eleman en fazla bir kez kullanılır. Soldan sağa dolaşılsaydı aynı eleman birden çok kez alınabilirdi (unbounded knapsack).

Not: O(n · S) çalışma süresi sözde-polinomdur (pseudo-polynomial): girdinin bit uzunluğuna göre değil, sayıların büyüklüğüne göre polinomdur. Problem genel olarak NP-tamdır; DP, toplam küçük olduğunda verimlidir.

8. Sonuç

Partition Equal Subset Sum, toplamın yarısını hedefleyen bir 0/1 Subset Sum problemine indirgenir. dp[i][j] tablosu her elemanı “al / alma” kararıyla değerlendirir, üstel kaba kuvvet çözümünü O(n · S) süreye indirir ve bellek O(S)'ye düşürülebilir.